

1. Project Goal

This project is a small but complete real-time data pipeline that simulates user engagement events (play, pause, finish, click), processes them in real time, stores them in a database, and generates simple analytics.

The main goal is to demonstrate how I think about building a data pipeline end to end: - Event generation - Real-time processing - Storage in a structured database - Reporting and analytics

The project was implemented as part of a technical evaluation for Thmanyah.

2. High-Level Architecture

The data flow is:

Event Producer → Kafka → Spark Structured Streaming → PostgreSQL → Analytics Reports

- A Python producer sends random engagement events to a Kafka topic. - Spark Structured Streaming reads from Kafka as a stream, parses JSON, and aggregates events in 1-minute windows. - PostgreSQL stores the events in a clean schema for analysis. - A Python analytics script runs SQL queries and prints key metrics in a readable format.

3. Technologies Used

- Kafka (Confluent image via Docker) Used as the streaming backbone to receive events in real time.

- Spark Structured Streaming (PySpark 3.4.1) Used to consume the Kafka stream, parse JSON, and compute aggregations over time windows.

- PostgreSQL Used as the main analytical store. Tables: - content - engagement_events

- Docker & Docker Compose Used to run Kafka, Zookeeper, PostgreSQL, and Redis in a reproducible way.

- Python Used for: - events_producer.py → sends events to Kafka - events_stream_job.py → Spark streaming job - analytics_report.py → analytics directly from PostgreSQL

4. How It Works (Short Walkthrough)

1) The producer generates random events that look like user engagement: - event_type: play, pause, finish, click - device: ios, android, chrome, web-safari - timestamps and durations

2) These events are sent to a Kafka topic named "engagement_stream".

3) Spark Structured Streaming reads from that topic, casts the Kafka value as JSON, and

applies a schema. Then it groups events by: - a 1-minute window on event_ts - content_id - event_type

4) The results are written to the console for easy inspection, and the raw events are stored in PostgreSQL for later analysis.

5) The analytics_report.py script connects to PostgreSQL and runs several SQL queries, for example: - number of events per event_type - top content by total engagement - completion rate (finish / play) per content - events per device

5. Design Choices and Trade-offs

- I used Kafka + Spark Structured Streaming to stay close to real-world data engineering patterns, even though the data itself is synthetic. - I kept the storage layer simple (PostgreSQL) instead of adding a data lake or warehouse, to keep the focus on the pipeline and streaming logic. - The streaming output is printed to the console rather than written to an external sink, to make it easy to run and evaluate without extra setup. - All scripts are triggered manually (no Airflow or scheduler) to keep the project small and easy to understand.

6. Limitations and Possible Extensions

Current limitations: - Events are random test data, not real production traffic. - No authentication, security, or multi-environment deployment. - Spark writes only to the console sink.

Possible future improvements: - Add a data lake (for example S3) to store raw and processed events. - Introduce a scheduler such as Airflow for regular batch jobs and orchestration. - Add monitoring and logging (Prometheus, Grafana). - Improve PostgreSQL schema with partitioning and indexing for larger volumes. - Connect the database to a BI tool (Looker, Metabase, or Power BI) for dashboards. - Expose the analytics via a small API layer.

7. Personal Note

This project was a good opportunity to connect multiple tools that I have been studying (Kafka, Spark, PostgreSQL, Docker) into one working pipeline.

While building it, I reinforced my understanding of: - real-time ingestion and processing, - schema design for analytics, - debugging across multiple containers and services.

I am actively learning and practicing data analytics, data engineering, Snowflake, and cloud data architectures. Joining a Saudi company like Thmanyah would be an honor for me, and I am confident in my ability to learn quickly, adapt, and contribute to real data products.

Thank you for your time and consideration.